

Практическая работа 10. Создание списка с прокруткой

В этом кодовом уроке вы узнаете, как сделать прокручиваемый список в приложении с помощью Jetpack Compose.

Вы будете работать с приложением Affirmations, которое отображает список affirmаций в сочетании с красивыми изображениями, чтобы принести позитив в ваш день!

Данные уже есть, вам нужно только взять их и отобразить в пользовательском интерфейсе.

Необходимые условия Знакомство со списками в Kotlin Опыт создания макетов с помощью Jetpack Compose Опыт запуска приложений на устройстве или эмуляторе Что вы узнаете Как создать карточку с материальным дизайном с помощью Jetpack Compose Как создать прокручиваемый список с помощью Jetpack Compose Что вы будете создавать Вы возьмете существующее приложение и добавите в его пользовательский интерфейс прокручиваемый список. Готовый продукт будет выглядеть следующим образом:

Что вам понадобится Компьютер с доступом в Интернет, веб-браузер и Android Studio Доступ к GitHub Загрузите начальный код В Android Studio откройте папку basic-android-kotlin-compose-training-affirmations.

URL стартового кода:

<https://github.com/google-developer-training/basic-android-kotlin-compose-training-affirmations>

Название ветки с кодом стартера: starter

Ожидается, что при сборке из кода ветки starter приложение будет отображать пустой экран.

2. Создание класса данных для элементов списка

Создание класса данных для affirmации В приложениях для Android списки состоят из элементов списка. Для отдельных элементов это может быть что-то простое, например строка или целое число. Для элементов списка, содержащих несколько частей данных, например изображение и текст, вам понадобится класс, содержащий все эти свойства. Классы данных - это тип классов, которые содержат только свойства, и могут предоставлять некоторые полезные методы для работы с этими свойствами.

Создайте новый пакет в папке com.example.affirmations.

Назовите новый пакет model. Пакет model будет содержать модель данных, которая будет представлена классом данных. Класс данных будет состоять из свойств, представляющих информацию, относящуюся к

«Утверждению», которое будет состоять из строкового ресурса и ресурса изображения. Пакеты - это каталоги, содержащие классы и даже другие каталоги.

Создайте новый класс в пакете `com.example.affirmations.model`.

Назовите новый класс `Affirmation` и сделайте его классом `Data`.

Name the new class `Affirmation` and make it a `Data` class.

`Affirmation.kt`

```
data class Affirmation(  
    val stringResourceId: Int,  
    val imageResourceId: Int  
)
```

Аннотируйте свойство `stringResourceId` с помощью аннотации `@StringRes` и аннотируйте `imageResourceId` с помощью аннотации `@DrawableRes`. Свойство `stringResourceId` представляет собой идентификатор текста аффирмации, хранящегося в строковом ресурсе. `imageResourceId` представляет собой идентификатор изображения аффирмации, хранящегося в ресурсе `drawable`.

`Affirmation.kt`

```
import androidx.annotation.DrawableRes  
import androidx.annotation.StringRes  
  
data class Affirmation(  
    @StringRes val stringResourceId: Int,  
    @DrawableRes val imageResourceId: Int  
)
```

В пакете `com.example.affirmations.data` откройте файл `Datasource.kt` и откомментируйте два оператора импорта и содержимое класса `Datasource`.

`Datasource.kt`

```
import com.example.affirmations.R  
import com.example.affirmations.model.Affirmation  
  
class Datasource() {  
    fun loadAffirmations(): List<Affirmation> {  
        return listOf<Affirmation>(  

```

```
        Affirmation(R.string.affirmation1, R.drawable.image1),
        Affirmation(R.string.affirmation2, R.drawable.image2),
        Affirmation(R.string.affirmation3, R.drawable.image3),
        Affirmation(R.string.affirmation4, R.drawable.image4),
        Affirmation(R.string.affirmation5, R.drawable.image5),
        Affirmation(R.string.affirmation6, R.drawable.image6),
        Affirmation(R.string.affirmation7, R.drawable.image7),
        Affirmation(R.string.affirmation8, R.drawable.image8),
        Affirmation(R.string.affirmation9, R.drawable.image9),
        Affirmation(R.string.affirmation10, R.drawable.image10))
    }
}
```

Примечание: Метод `loadAffirmations()` собирает все данные, предоставленные в начальном коде, и возвращает их в виде списка. Позже вы будете использовать его для создания прокручиваемого списка.

3. Добавьте список в приложение

Создайте карточку со списком. Это приложение предназначено для отображения списка affirmаций. Первым шагом в настройке пользовательского интерфейса для отображения списка является создание элемента списка. Каждый элемент affirmации состоит из изображения и строки. Данные для каждого из этих элементов поставляются вместе со стартовым кодом, и вам предстоит создать компонент пользовательского интерфейса для отображения такого элемента.

Элемент будет состоять из композита `Card`, содержащего композиты `Image` и `Text`. В Compose карточка - это поверхность, которая отображает содержимое и действия в одном контейнере. В предварительном просмотре карточка `Affirmation` будет выглядеть следующим образом:

На карточке изображено изображение с текстом под ним. Этого вертикального расположения можно добиться с помощью композита `Column`, обернутого в композит `Card`. Вы можете попробовать сделать это самостоятельно или выполнить следующие шаги.

Откройте файл `MainActivity.kt`. Создайте новый метод под методом `AffirmationsApp()`, назвав его `AffirmationCard()`, и аннотируйте его аннотацией `@Composable`. `MainActivity.kt`

```
@Composable
fun AffirmationsApp() {
}

@Composable
fun AffirmationCard() {
}
}
```

Отредактируйте сигнатуру метода, чтобы он принимал в качестве параметра объект `Affirmation`. Объект `Affirmation` берется из пакета `model`.

MainActivity.kt

```
import com.example.affirmations.model.Affirmation

@Composable
fun AffirmationCard(affirmation: Affirmation) {

}
```

Добавьте в подпись параметр-модификатор. Установите для параметра значение `Modifier` по умолчанию.

MainActivity.kt

```
@Composable
fun AffirmationCard(affirmation: Affirmation, modifier: Modifier = Modifier) {

}
```

Примечание: Лучше всего передавать модификатор в каждый композит и устанавливать его значение по умолчанию.

Внутри метода `AffirmationCard` вызовите композит `Card`. Передайте параметр модификатора.

MainActivity.kt

```
import androidx.compose.material3.Card

@Composable
fun AffirmationCard(affirmation: Affirmation, modifier: Modifier = Modifier) {
    Card(modifier = modifier) {

    }
}
```

Добавьте композитный столбец внутри композитного столбца `Card`. Элементы в композите `Column` располагаются в пользовательском интерфейсе вертикально. Это позволяет поместить изображение над связанным с ним текстом. И наоборот, композит `Row` располагает содержащиеся в нем элементы горизонтально. MainActivity.kt

```
import androidx.compose.foundation.layout.Column

@Composable
```

```
fun AffirmationCard(affirmation: Affirmation, modifier: Modifier = Modifier) {
    Card(modifier = modifier) {
        Column {

        }
    }
}
```

Добавьте Image composable внутрь лямбда-тела Column composable. Напомним, что Image composable всегда требует ресурс для отображения и contentDescription. Ресурсом должен быть ресурс painterResource, передаваемый параметру painter. Метод painterResource загружает либо векторные рисунки, либо растрованные форматы активов, например PNG. Кроме того, для параметра contentDescription передайте ресурс stringResource.

MainActivity.kt

```
import androidx.compose.foundation.Image
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource

@Composable
fun AffirmationCard(affirmation: Affirmation, modifier: Modifier = Modifier) {
    Card(modifier = modifier) {
        Column {
            Image(
                painter = painterResource(affirmation.imageResourceId),
                contentDescription = stringResource(affirmation.stringResourceId),
            )
        }
    }
}
```

В дополнение к параметрам painter и contentDescription передайте модификатор и contentScale. ContentScale определяет, как изображение должно быть масштабировано и отображено. У объекта Modifier должен быть установлен атрибут fillMaxWidth и высота 194.dp. Масштаб содержимого должен иметь значение ContentScale.Crop.

MainActivity.kt

```
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.ui.unit.dp
import androidx.compose.ui.layout.ContentScale

@Composable
fun AffirmationCard(affirmation: Affirmation, modifier: Modifier = Modifier) {
    Card(modifier = modifier) {
```

```

        Column {
            Image(
                painter = painterResource(affirmation.imageResourceId),
                contentDescription = stringResource(affirmation.stringResourceId),
                modifier = Modifier
                    .fillMaxWidth()
                    .height(194.dp),
                contentScale = ContentScale.Crop
            )
        }
    }
}

```

Внутри колонки создайте компонент `Text` после компонента `Image`. Передайте в параметр `text` строковый ресурс `affirmation.stringResourceId`, передайте объект `Modifier` с атрибутом `padding`, установленным на `16.dp`, и задайте тему текста, передав в параметр `style` материал `MaterialTheme.typography.headlineSmall`.

MainActivity.kt

```

import androidx.compose.material3.Text
import androidx.compose.foundation.layout.padding
import androidx.compose.ui.platform.LocalContext

@Composable
fun AffirmationCard(affirmation: Affirmation, modifier: Modifier = Modifier) {
    Card(modifier = modifier) {
        Column {
            Image(
                painter = painterResource(affirmation.imageResourceId),
                contentDescription = stringResource(affirmation.stringResourceId),
                modifier = Modifier
                    .fillMaxWidth()
                    .height(194.dp),
                contentScale = ContentScale.Crop
            )
            Text(
                text =
                    LocalContext.current.getString(affirmation.stringResourceId),
                modifier = Modifier.padding(16.dp),
                style = MaterialTheme.typography.headlineSmall
            )
        }
    }
}

```

Предварительный просмотр композитной карты `AffirmationCard` Карточка - это основа пользовательского интерфейса приложения `Affirmations`, и вы приложили немало усилий для ее создания! Чтобы проверить, правильно ли выглядит карточка, вы можете создать композит, который можно предварительно просмотреть, не запуская все приложение.

Создайте приватный метод `AffirmationCardPreview()`. Аннотируйте метод с помощью `@Preview` и `@Composable`. `MainActivity.kt`

```
import androidx.compose.ui.tooling.preview.Preview

@Preview
@Composable
private fun AffirmationCardPreview() {

}
```

Внутри метода вызовите компонент `AffirmationCard` и передайте ему новый объект `Affirmation` с ресурсом `R.string.affirmation1` string и ресурсом `R.drawable.image1` drawable, переданными в его конструктор.

`MainActivity.kt`

```
@Preview
@Composable
private fun AffirmationCardPreview() {
    AffirmationCard(Affirmation(R.string.affirmation1, R.drawable.image1))
}
```

Откройте вкладку Split, и вы увидите предварительный просмотр `AffirmationCard`. При необходимости нажмите `Build & Refresh` на панели Design, чтобы отобразить предварительный просмотр.

Создание списка Компонент элемента списка - это строительный блок списка. Создав элемент списка, вы можете использовать его для создания самого компонента списка.

Создайте функцию `AffirmationList()`, аннотируйте ее аннотацией `@Composable` и объявите список объектов `Affirmation` в качестве параметра в сигнатуре метода. `MainActivity.kt`

```
@Composable
fun AffirmationList(affirmationList: List<Affirmation>) {

}
```

Объявите объект модификатора в качестве параметра в сигнатуре метода со значением по умолчанию `Modifier`.

`MainActivity.kt`

```
@Composable
fun AffirmationList(affirmationList: List<Affirmation>, modifier: Modifier =
Modifier) {

}
```

В Jetpack Compose прокручиваемый список можно создать с помощью композита `LazyColumn`. Разница между `LazyColumn` и `Column` заключается в том, что `Column` следует использовать, когда у вас есть небольшое количество элементов для отображения, поскольку Compose загружает их все сразу. Колонка может содержать только predetermined, или фиксированное, количество элементов. `LazyColumn` может добавлять содержимое по требованию, что делает его удобным для длинных списков и особенно для тех случаев, когда длина списка неизвестна. `LazyColumn` также обеспечивает прокрутку по умолчанию, без дополнительного кода. Объявите `LazyColumn` композитным внутри функции `AffirmationList()`. Передайте объект модификатора в качестве аргумента `LazyColumn`.

MainActivity.kt

```
import androidx.compose.foundation.lazy.LazyColumn

@Composable
fun AffirmationList(affirmationList: List<Affirmation>, modifier: Modifier =
Modifier) {
    LazyColumn(modifier = modifier) {

    }
}
```

В теле лямбда-функции `LazyColumn` вызовите метод `items()` и передайте в него `affirmationList`. Метод `items()` - это то, как вы добавляете элементы в `LazyColumn`. Этот метод несколько уникален для данного `composable`, и в остальном не является обычной практикой для большинства `composable`.

MainActivity.kt

```
import androidx.compose.foundation.lazy.items

@Composable
fun AffirmationList(affirmationList: List<Affirmation>, modifier: Modifier =
Modifier) {
    LazyColumn(modifier = modifier) {
        items(affirmationList) {

        }
    }
}
```


Вызов метода `items()` требует лямбда-функции. В этой функции укажите параметр `affirmation`, который представляет один элемент утверждения из списка `affirmationList`. `MainActivity.kt`

```
@Composable
fun AffirmationList(affirmationList: List<Affirmation>, modifier: Modifier =
Modifier) {
    LazyColumn(modifier = modifier) {
        items(affirmationList) { affirmation ->

        }
    }
}
```

Для каждого утверждения в списке вызовите составную `AffirmationCard()`. Передайте ему аффирмацию и объект `Modifier` с атрибутом `padding`, установленным на `8.dp`. `MainActivity.kt`

```
@Composable
fun AffirmationList(affirmationList: List<Affirmation>, modifier: Modifier =
Modifier) {
    LazyColumn(modifier = modifier) {
        items(affirmationList) { affirmation ->
            AffirmationCard(
                affirmation = affirmation,
                modifier = Modifier.padding(8.dp)
            )
        }
    }
}
```

Отображение списка В составном `AffirmationsApp` получите текущие направления компоновки и сохраните их в переменной. Они будут использоваться для настройки подкладки позже. `MainActivity.kt`

```
import com.example.affirmations.data.Datasource

@Composable
fun AffirmationsApp() {
    val layoutDirection = LocalLayoutDirection.current
}
```

Теперь создайте композит `Surface`. Этот композит будет задавать подложку для композита `AffirmationsList`. `MainActivity.kt`

```
import com.example.affirmations.data.Datasource

@Composable
```

```
fun AffirmationsApp() {
    val layoutDirection = LocalLayoutDirection.current
    Surface() {
    }
}
```

Передайте композиту Surface модификатор, который заполняет максимальную ширину и высоту родителя, устанавливает отступы строки состояния и задает начальный и конечный отступы для layoutDirection. Вот пример того, как преобразовать объект LayoutDirection в padding: `WindowInsets.safeDrawing.asPaddingValues().calculateStartPadding(layoutDirection)`.

MainActivity.kt

```
import com.example.affirmations.data.Datasource

@Composable
fun AffirmationsApp() {
    val layoutDirection = LocalLayoutDirection.current
    Surface(
        Modifier = Modifier
            .fillMaxSize()
            .statusBarsPadding()
            .padding(
                start = WindowInsets.safeDrawing.asPaddingValues()
                    .calculateStartPadding(layoutDirection),
                end = WindowInsets.safeDrawing.asPaddingValues()
                    .calculateEndPadding(layoutDirection),
            ),
    ) {
    }
}
```

В лямбде для Surface composable вызовите AffirmationList composable и передайте DataSource().loadAffirmations() в параметр affirmationList.

Примечание: класс DataSource находится в пакете data.

MainActivity.kt

```
import com.example.affirmations.data.Datasource

@Composable
fun AffirmationsApp() {
    val layoutDirection = LocalLayoutDirection.current
    Surface(
        Modifier = Modifier
            .fillMaxSize()
            .statusBarsPadding()
            .padding(
```

```
        start = WindowInsets.safeDrawing.asPaddingValues()
            .calculateStartPadding(layoutDirection),
        end = WindowInsets.safeDrawing.asPaddingValues()
            .calculateEndPadding(layoutDirection),
    ),
) {
    AffirmationsList(
        affirmationList = Datasource().loadAffirmations(),
    )
}
}
```

Запустите приложение Affirmations на устройстве или эмуляторе и посмотрите на готовый продукт!

5. Заключение

Теперь вы знаете, как создавать карточки, элементы списка и прокручиваемые списки с помощью Jetpack Compose! Не забывайте, что это лишь базовые инструменты для создания списка. Вы можете дать волю своему творчеству и настроить элементы списка так, как вам нравится!

Резюме

Используйте составные элементы карточки для создания элементов списка. Модифицируйте пользовательский интерфейс, содержащийся в композите Card. Создание прокручиваемого списка с помощью композита LazyColumn. Создайте список, используя пользовательские элементы списка.